

Distillation

One idea, four methods — logits, on-policy, black-box, and reasoning traces — and how to pick the right one

You have a 32B model that solves competition math. Your serving budget has room for a 7B. The obvious plan — run the same reinforcement-learning recipe that made the 32B smart, but on the small model — is the plan DeepSeek actually tried, and published: large-scale RL on a 32B base reached 47.0% on AIME 2024. Then they tried something that looks almost too simple to work: fine-tune the same small models on 800K outputs *sampled from the big one*. No RL, no reward model, no rollout infrastructure. The distilled 32B scored 72.6% — twenty-five points higher than the RL version, at a fraction of the cost — and the distilled 7B outscored models many times its size.¹

That result is why this chapter exists as a chapter, and not as three scattered sections. Distillation — training one model on the outputs of another — has quietly become the main way capability actually reaches the models people deploy. The R1 family that most people ran locally in 2025 were distillations. Qwen3's small models are distillations. Gemma's small models are distillations. Llama 3.2's 1B and 3B are prune-then-distill. When you use a good small model today, you are almost always using a distillate of something bigger.¹²³⁴

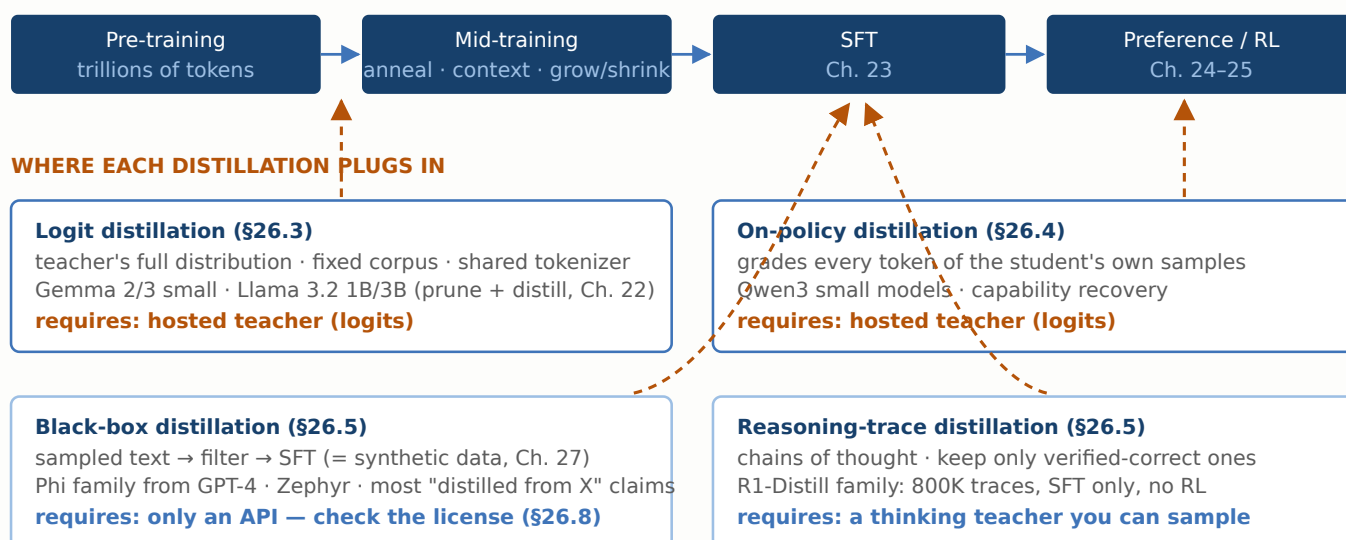
The catch — and the reason this topic confuses people — is that "distillation" names at least four different procedures with different requirements, different costs, and different failure modes. Two of them need a teacher whose logits you can read, which means a teacher you host. One works through an API. They plug into different stages of the training pipeline, from pre-training to the final post-training pass. Pick the wrong one and you will either burn compute you didn't need to spend or hit a wall you can't fix — asking an API teacher for its probability distribution, say, when all it will ever give you is text. By the end of this chapter you will be able to name which of the four you need from the shape of your problem: what you want (smaller model? specific skill?), what you have (the teacher's weights, or just its chat endpoint?), and where in the pipeline you are standing.

26.1 Four things called distillation

Every distillation has the same cast: a **teacher** that is better (usually bigger), a **student** you are training, and a **signal** that flows from one to the other. Everything that matters is decided by two questions about that signal. First: *what does the teacher emit* — its full probability distribution over the vocabulary (logits), or only sampled text? Second: *whose text is being graded* — text from a fixed corpus or the teacher (off-policy), or text the student itself generated (on-policy)? Cross the two questions and you get the four methods, each with its natural home in the pipeline you have been building since Chapter 5:

- **Logit distillation** (§26.3): the student reads the teacher's full next-token distribution at every position of a fixed corpus. The densest signal there is; needs a hosted teacher *and* a shared tokenizer. Lives in pre-training and mid-training — this is how Gemma's and Llama 3.2's small models are made, and it pairs with the pruning and upcycling of Chapter 22.
- **On-policy distillation** (§26.4): the student writes; the teacher grades *every token of the student's own text* against its full distribution. Dense signal on exactly the states the student visits. Lives in post-training, where it competes directly with the RL of Chapter 25 — and often wins on cost.
- **Black-box distillation** (§26.5): the teacher only provides sampled text, which you filter and fine-tune on. This is what "distilling GPT-4/Claude/Gemini" means: it is synthetic-data SFT wearing a different name, which is why its data-side story lives in Chapter 27 and its licensing traps in §26.8.
- **Reasoning-trace distillation** (§26.5): black-box distillation specialized to chains of thought — x. The R1 recipe above. The cheapest route to a small model that reasons.

THE PIPELINE (Chapter 5)



Blue-bordered boxes need the teacher's logits (you host it). Light-bordered boxes need only samples (an API is enough).

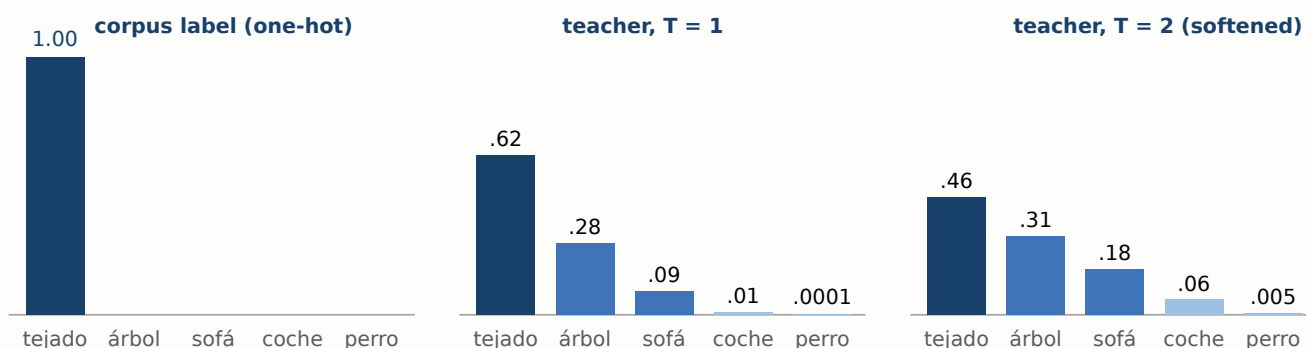
Figure 26.1 — Four methods, one pipeline. Logit distillation feeds pre- and mid-training; the other three are post-training moves that land on the SFT stage (black-box, traces) or compete with RL (on-policy). The dividing line that matters most in practice is access: the left column of each pair needs a teacher whose distributions you can read.

Keep Figure 26.1 in your head as the chapter's map. We now build the machinery once (§26.2) — soft labels, temperature, and the two directions of KL divergence — because every method in this chapter is that machinery pointed at a different data stream.

26.2 The machinery: soft labels and two directions of KL

Recall what ordinary training does (Chapter 2): at each position, the corpus says "the next token was *tejado*," and cross-entropy pushes the model's distribution toward a one-hot spike on that token. One token of truth, zero information about

everything else. Now look at what a good teacher knows at that same position. Give a 70B model the prefix "*El gato se subió al*" and read its full next-token distribution:



Prefix: "El gato se subió al ...". Teacher logits (7.1, 6.3, 5.2, 3.0, -2.0) at each temperature; 5 of 128K vocab entries shown.

Figure 26.2 — Dark knowledge. The corpus label says only "tejado". The teacher additionally says that *árbol* is plausible (0.28), *sofá* possible (0.09), *coche* unlikely and *perro* absurd — the relative geometry of the whole language at this position. Raising the temperature to 2 flattens the distribution, amplifying exactly those informative ratios in the training signal.

The corpus taught you one fact. The teacher teaches you a *ranking of all counterfactuals* — cats climb trees nearly as often as roofs, never onto dogs — and this is at every single position. Hinton, Vinyals and Dean called it **dark knowledge**: the information carried by the small probabilities, invisible in the argmax, and it is why a student trained on soft labels learns faster per token than one trained on text alone.⁵

Matching distributions: forward KL

Training the student to match the teacher means minimizing a divergence between two distributions at each position. With teacher p and student q_θ , the classic choice is the **forward KL divergence**:

$$L_{\text{KD}} = \text{KL} (p \parallel q_\theta) = \sum_{v \in V} p(v) \log \frac{p(v)}{q_\theta(v)}$$

Since the teacher term is constant with respect to θ , the gradient is exactly that of cross-entropy with soft targets: $-\sum_v p(v) \nabla_\theta \log q_\theta(v)$. Ordinary training is this loss with p collapsed to one-hot — distillation just un-collapses it. Hinton's recipe adds a temperature τ applied to *both* models' logits, $p^{(\tau)}(v) \propto \exp(z_v/\tau)$, and multiplies the loss by τ^2 so gradient magnitudes stay comparable as you soften.⁵ The numbers in Figure 26.2 show what $\tau = 2$ buys: the ratio structure among the plausible-but-not-chosen tokens becomes a larger share of the signal.

The direction matters: forward vs. reverse

KL is asymmetric, and the asymmetry is not a technicality — it decides the student's personality. Forward KL, $\text{KL}(p \parallel q)$, is weighted by the *teacher's* probabilities: the student is punished wherever the teacher has mass and it doesn't, so it must spread itself to **cover every mode** the teacher has. A small student covering a large teacher ends up assigning middling probability everywhere — including to regions between the teacher's modes where neither model should ever sample. Reverse KL, $\text{KL}(q \parallel p)$, is weighted by the *student's* probabilities: the student is punished where it

puts mass and the teacher doesn't, so its optimal move is to pick the modes it can afford and match them sharply — **mode-seeking**. For a generator that will autoregressively sample its own text, quality lives in "never say things the teacher considers wrong," which is precisely what reverse KL optimizes; this observation drives MiniLLM and the on-policy family of §26.4.⁶⁷

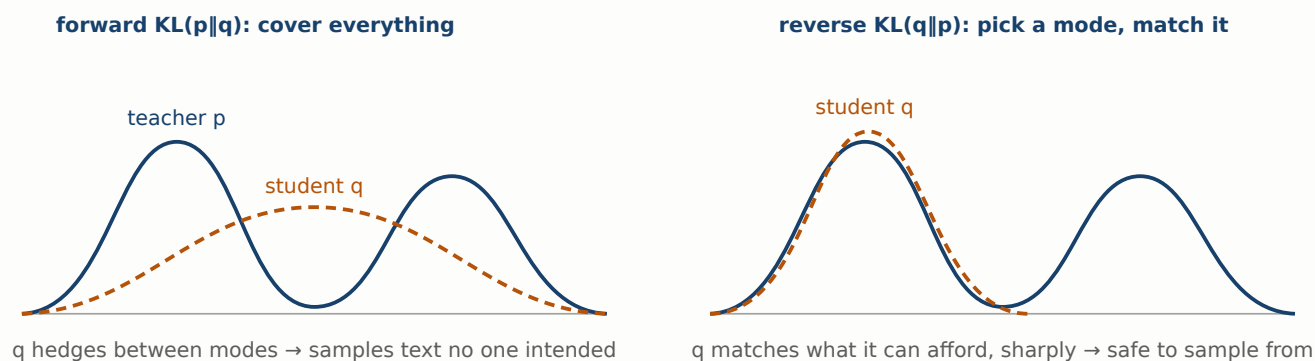


Figure 26.3 — Why the KL direction is a design decision. A capacity-limited student cannot be the teacher everywhere. Forward KL forces mass-covering — fine when the loss is averaged over a fixed corpus (§26.3), where the data distribution keeps the student honest. Reverse KL forces mode-seeking — what you want when the student will generate autoregressively, because hedged mass between modes becomes sampled nonsense.

Now the whole chapter compresses into one sentence: *logit distillation is forward KL on a fixed corpus; on-policy distillation is reverse KL on the student's own samples; black-box distillation is forward KL against a one-sample estimate of the teacher (its text), which is just cross-entropy — SFT.* Everything else is engineering. To the engineering, then.

26.3 Logit distillation: pre-training with a better label

The heavyweight use of distillation happens where you might least expect it: *during pre-training itself*. Instead of training a small model from scratch on next-token labels, train it on a large teacher's full distribution over the same corpus. Gemma is the clearest public case: the 9B model was trained on 8 trillion tokens of teacher distributions — the paper argues this "simulates" training on far more tokens than you have, because each position carries the full ranking of Figure 26.2 rather than one label.³ The loss per position is the forward KL above, usually mixed with the ordinary cross-entropy:

$$L = \alpha \tau^2 \text{KL} \left(p_T^{(\tau)} \parallel q_\theta^{(\tau)} \right) + (1 - \alpha) L_{\text{CE}}(y, q_\theta), \quad \alpha \in [0.5, 1]$$

and the implementation is small enough to read in full:

```
# logit distillation: the student learns the teacher's whole distribution
import torch.nn.functional as F

def kd_loss(student_logits, teacher_logits, T=2.0):
    # soften both, compare, and rescale by T^2 (Hinton et al., 2015)
```

```

log_q = F.log_softmax(student_logits / T, dim=-1)
p      = F.softmax(teacher_logits / T, dim=-1)
return F.kl_div(log_q, p, reduction="batchmean") * T * T

for batch in corpus:                                # the same stream as pre-training
    with torch.no_grad():
        t_logits = teacher(batch).logits  # teacher inference, no grads
        s_logits = student(batch).logits
        loss = 0.7 * kd_loss(s_logits, t_logits) \
              + 0.3 * F.cross_entropy(s_logits.transpose(1, 2), batch.labels)
        loss.backward(); opt.step(); opt.zero_grad()
# full training scripts: github.com/omar-florez/training-efficient-llms

```

What it costs, honestly

The line `teacher(batch)` hides the entire cost model. Distilling over D tokens adds one teacher forward pass per token: about $2 N_T D$ FLOPs on top of the student's $6 N_S D$ (Chapter 6's forward/backward accounting). For a 27B teacher over a 9B student, that is $2 \cdot 27 / (6 \cdot 9) = 1.0$ — the teacher *doubles* your compute per token. So why is this efficient? Because of what it buys per token: Gemma 2's 9B outperformed a from-scratch model of its size by wide margins, and NVIDIA's Minitron showed the sharpest version of the accounting — prune a 15B model down to 8B, heal it with logit distillation, and you reach the quality of a from-scratch 8B using **up to 40× fewer training tokens**, an $\sim 1.8\times$ compute saving on the whole model family.⁸ That prune-then-distill recipe is exactly how Llama 3.2's 1B and 3B were produced from the 8B.⁴ When the teacher already exists — and it always does, you trained it — its inference is the cheapest quality-per-token you can buy for a small model.

Two practical notes the papers agree on. You might hope to pay the teacher once and cache its outputs; the arithmetic kills it. Storing even a top-128 truncation of the distribution (128 probabilities + 128 token ids) for a 100B-token run is roughly $100 \times 10^9 \times 128 \times 6 \text{ bytes} \approx 77 \text{ TB}$ — so in practice the teacher runs *online*, alongside the student, and you engineer its inference like a serving problem (batched, KV-less, bf16). And the whole section assumes teacher and student share a tokenizer: forward KL compares distributions *over the same vocabulary*, position by position. Different tokenizers mean different vocabularies *and* different token boundaries, and the loss stops type-checking — the pitfall list of §26.8 returns to this, because it is the most common way a distillation plan dies on day one.

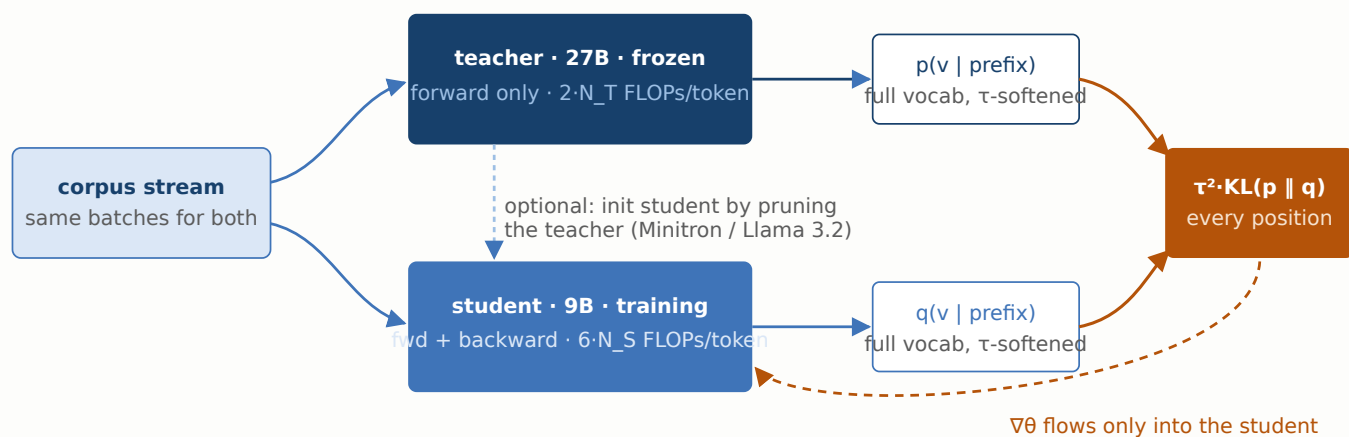


Figure 26.4 — Logit distillation as a training loop. Teacher and student read the identical corpus stream; their τ -softened distributions meet in a forward-KL loss at every position; gradients flow only into the student. The dashed vertical arrow is the Chapter 22 connection: initializing the student by pruning the teacher (depth or width) before distilling is what turned 40× fewer tokens into from-scratch quality for Minitron and Llama 3.2.⁸⁴

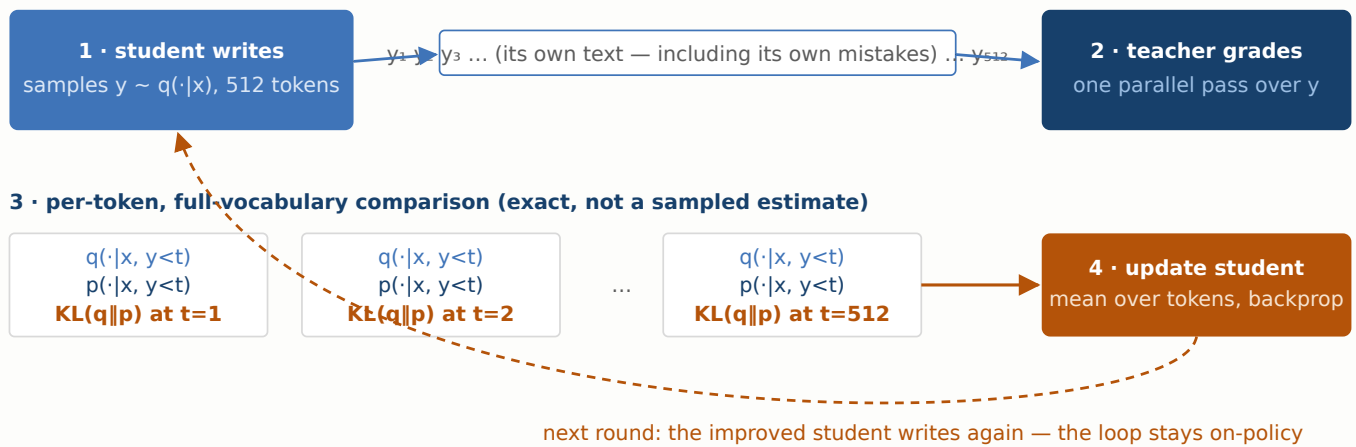
26.4 On-policy distillation: the teacher grades the student's own work

Logit distillation has a blind spot, and you have met it before under the name *exposure bias*. The student learns to imitate the teacher on prefixes drawn from the corpus — text that is, by construction, good. But at inference the student conditions on *its own* previous tokens. The first small mistake produces a prefix the student never trained on; its behavior there is unconstrained; errors compound token by token. The student has only ever studied worked examples of other people's essays, and now it must finish its own.⁷

The fix is the same one RL uses: train on the states the student actually visits. **On-policy distillation** flips who writes and who grades. The student generates a completion to a prompt — its own text, mistakes and all. The teacher then computes its full distribution at *every position of the student's sample*, and the student minimizes the per-token reverse KL against it:

$$\mathcal{L}_{\text{OPD}}(\theta) \coloneqq \mathbb{E}_{y \sim q_{\theta}(\cdot \mid x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} \text{KL}(\text{Big}(q_{\theta}(\cdot \mid x, y_{\leq t})) \parallel \text{Big}(p_T(\cdot \mid x, y_{\leq t})) \right]$$

Read it carefully, because each piece was chosen in §26.2. The expectation is over the *student's* samples — on-policy, so the loss lives exactly where the student's errors live, including the strange prefixes only it produces. The divergence is *reverse* KL — mode-seeking, so the student sharpens onto behavior the teacher endorses instead of hedging. And crucially, each term is *exact*, not estimated: given the prefix, both distributions over the vocabulary are available in one forward pass each, so every token yields a full-vocabulary comparison. This is the GKD recipe of Agarwal et al., and the engine behind Qwen3's small models.⁷²



Contrast with RL (Ch. 25): same rollout, but there the grader returns ONE scalar for all 512 tokens.

Figure 26.5 — The on-policy distillation loop. The student writes, the teacher grades every token of the student's own text against its full distribution, and the mean per-token reverse KL updates the student. The teacher's step is a single parallel forward pass (a prefill, in Chapter 13's terms) — it never has to *generate* anything.

Why it is cheap — the arithmetic that beats RL

Compare this loop with the GRPO loop of Chapter 25 on the two axes that dominate cost there: rollouts and signal. The rollout is identical — the student generates, autoregressively, one token at a time; for an 8B student and a 512-token completion that is 512 serial forward passes, ~8 TFLOPs, and (Chapter 13) a memory-bound crawl. But now count what each framework extracts from that rollout. RL boils it down to **one scalar reward**, and GRPO needs a *group* of rollouts per prompt (typically 8–16) just to turn that scalar into a usable advantage estimate. On-policy distillation extracts **512 full distributions** — a dense, per-token, exact supervision signal — from a *single* rollout, and the teacher's grading costs one parallel pass (~72 TFLOPs for a 70B teacher over 512 tokens) that runs at prefill efficiency, not decode efficiency. No reward model to train, no value baseline, no group sampling, no reward hacking to police. The published numbers land where the arithmetic points: Qwen3's team reports that distilling its small models from the flagship needed roughly **1/10 the GPU-hours of the RL pipeline** it replaced, with better results; Thinking Machines' reproduction reports similar order-of-magnitude savings on capability-recovery tasks.²⁹

The loop is also short enough to read whole:

```
# on-policy distillation step (GKD-style): student writes, teacher grades
prompts = next(prompt_loader)                                # e.g. math problems, chats
with torch.no_grad():
    y = student.generate(prompts, max_new_tokens=512,
                          do_sample=True, temperature=1.0)    # ON-policy: its own
text
    p_log = F.log_softmax(teacher(y).logits / T, dim=-1)      # one parallel pass

s_log = F.log_softmax(student(y).logits / T, dim=-1)
q      = s_log.exp()
rkl    = (q * (s_log - p_log)).sum(-1)                       # exact reverse KL per token
loss   = rkl[completion_mask(y, prompts)].mean()            # grade only generated tokens
loss.backward(); opt.step(); opt.zero_grad()
```

A worked micro-step

To make the loss concrete, run one position of the loop by hand. The student, mid-sample, has just written "...the derivative of" and its distribution over the next token puts 0.50 on `x2`, 0.30 on `the`, 0.15 on `sin`, 0.05 on everything else. The teacher, given the same prefix, puts 0.72 on `the`, 0.20 on `sin`, 0.05 on `x2`, 0.03 elsewhere. The per-token reverse KL is

$$\text{KL}(q\|p) = 0.50\ln\frac{0.50}{0.05} + 0.30\ln\frac{0.30}{0.72} + 0.15\ln\frac{0.15}{0.20} + \dots \approx 1.15 - 0.26 - 0.04 + \dots \approx 0.85$$

Look at where the loss comes from: almost all of it is the first term — the student's confident 0.50 on a token the teacher rates at 0.05. The gradient therefore pushes *down* hardest exactly on the student's overconfident mistake, and mildly *up* on the teacher-endorsed `the`. Sum that over 512 positions of the student's own text, and you have surgical, per-token correction of the places where this student, on this prefix, goes wrong — supervision no fixed corpus could have provided, because no corpus contains this prefix. Compare the RL version of the same moment: if the episode's final answer came out wrong, *every* token in it — including the good ones — receives the same negative scalar. Distillation blames the token that deserves it; RL blames the sentence.

Hyperparameters, briefly, since the loop has few: sample at temperature ≈ 1 (you want the student's true distribution, warts included — cooling the sampling hides the mistakes you came to fix); keep the distillation temperature $\tau = 1$ for reverse KL (softening matters less than in §26.3 because the signal is already dense on-policy); learning rates sit where SFT's do, not RL's (this is a supervised loss — no trust regions, no KL-to-reference penalty needed, because the loss is a KL to something trustworthy); and a few hundred to a few thousand steps often suffice for behavior transfer — Qwen3 reports its distillation phase in thousands of GPU-hours where the RL it replaced took tens of thousands.²

When do you reach for it? Three situations recur. **Making a small model from a big one you own** — the Qwen3 pattern: SFT the student on teacher outputs first (off-policy warm start, §26.5), then on-policy distill; the warm start gets it into the teacher's neighborhood, the on-policy phase fixes the states SFT never visits. **Capability recovery** — you quantized (Chapter 15), pruned (Chapter 22), or merged a model and it got slightly worse everywhere; on-policy distillation against the original model is the cheapest repair, because the "teacher" is just the pre-surgery checkpoint. **Continual learning** — you mid-trained a deployed model on new domain data (Chapter 19) and its instruction-following regressed; distilling from its own earlier post-trained self restores behavior without re-running the whole preference pipeline. In all three, the teacher check that gates everything: *same tokenizer, weights you can run*. If either fails, you are in §26.5's world.

26.5 Black-box distillation and reasoning traces: when all you have is text

Now remove the logits. The teacher is a frontier model behind an API: you send prompts, you get text. Whatever the marketing says, there is no distribution to match — the only supervision left is to treat the teacher's sampled text as

ground truth and do cross-entropy on it. Formally, that is forward-KL distillation where the teacher's distribution has been replaced by a single sample from it (sequence-level knowledge distillation, in Kim & Rush's original framing¹⁰); operationally, it is SFT on synthetic data, and everything Chapter 27 says about generation, filtering, and rejection sampling applies verbatim. The signal is ~17 bits per token (one token id from a 128K vocabulary) instead of a full distribution — you compensate with volume and with selection, keeping only the teacher's best outputs. This is the recipe behind the Phi family (textbook-style synthetic data from GPT-4-class teachers¹¹), Zephyr (distilled SFT + distilled preferences¹²), and most open models that advertise a frontier parent.

Reasoning traces are black-box distillation's power move, and the numbers from this chapter's opening deserve their full table. DeepSeek generated 800K samples with R1 — 600K long chains of thought, filtered so only traces reaching verified-correct answers survive, plus 200K general samples — and ran plain SFT on open student bases. No logits, no RL on the students, no reward model:¹

Table 26.1 — Distillation vs. RL on the same student (DeepSeek-R1 report, pass@1).¹

STUDENT / METHOD	AIME 2024	MATH- 500	POST-TRAINING COMPUTE
Qwen-32B + large-scale RL (R1-Zero recipe)	47.0	91.6	thousands of GPU-hrs of rollouts
Qwen-32B + SFT on 800K R1 traces	72.6	94.3	one SFT pass
Qwen-7B + SFT on the same traces	55.5	92.8	one SFT pass

Twenty-five points on AIME, in favor of the *cheaper* method. The paper's own conclusion is the one to teach: the reasoning patterns a huge model discovers through RL *transfer through text* — the trace is a serialization of the capability — while running RL directly on a small base model both costs more and achieves less, because the small model lacks the base capability for RL's sparse signal to amplify. Distill first; RL afterwards, on top of the distillate, if you still need the last few points. One honest caveat travels with the table: trace-distilled students inherit the teacher's *style* of reasoning without its robustness — they imitate the shape of deliberation and can wobble on out-of-distribution problems, which is why §26.8 insists you evaluate distilled reasoners beyond the teacher's favorite benchmarks.

What a trace run actually costs

Put numbers on the R1-scale recipe so you can budget your own. Suppose 600K reasoning traces averaging ~4K tokens: that is ~2.4B *generated* tokens — but generation is only the visible cost. To filter by verified answers you over-sample: at 4 candidate traces per problem with a ~50% survival rate after answer-checking, dedup, and format/language filters, you generate closer to 5–10B tokens to keep 2.4B. Self-hosting the teacher, that is a serving problem you can price with Chapter 13's arithmetic — decode-bound, so cache size and batch shape set your throughput; a GQA-8 70B-class teacher on one 8-GPU node sustains roughly 10⁴ tokens/s in large-batch decode, putting 10B tokens at ~10–12 node-days. Through an API, the same tokens are priced per million on the provider's meter — often the difference between "a weekend and a node" and "a purchase order." Either way, notice what the money buys: the expensive artifact is the *dataset*, not any training run. It is reusable across every student size you will ever build (DeepSeek fine-tuned six students from the same 800K), it version-controls like data because it *is* data (Chapter 10's traceability

discipline applies), and its quality ceiling is set by your filters, not your FLOPs — which is why the filtering stage deserves the engineering attention teams instinctively give to the training loop instead.

For a bilingual model the recipe carries one extra, non-optional filter: **language**. Reasoning teachers drift into English mid-trace even when prompted in Spanish — the internal monologue falls back to the language the capability was trained in. A student distilled on those traces learns to abandon Spanish exactly when the problem gets hard, which for a Spanish-first model is failure, not quirk. Filter traces for language consistency (a fastText-style classifier per span is enough), keep the final-answer language matched to the prompt, and report reasoning benchmarks per language — the Spanish AIME-equivalent score is the one your users experience, and it will lag the English number until the trace mix forces it not to.

The practical pipeline is four steps, and every step is a Chapter 27 topic wearing reasoning clothes: *prompt* a thinking teacher with problems that have checkable answers; *sample* generously (multiple traces per problem, high temperature); *filter* hard — verify final answers, drop mixed-language and malformed traces, dedupe near-copies (Chapter 10's machinery); *SFT* with the trace in the loss and, usually, a length cap so you don't teach unbounded rambling. The costs are API tokens and filtering throughput, not GPUs — which is exactly why licensing, not compute, is the binding constraint here, and why §26.8 opens with it.

26.6 Choosing the signal: the comparison that settles arguments

You now have all four methods. Here is the chapter in one table — the axes from §26.1, the machinery from §26.2, the costs from the three sections since:

Table 26.2 — The four distillations, side by side. "Signal per token" is what the student learns from at each position of the training stream.

METHOD	WHOSE TEXT	SIGNAL PER TOKEN	LOSS	TEACHER ACCESS	PIPELINE STAGE	BEST AT
Logit (§26.3)	corpus	full distribution	forward KL, τ^2	hosted + same tokenizer	pre / mid-training	building small models; healing pruned ones
On-policy (§26.4)	student's own	full distribution	reverse KL	hosted + same tokenizer	post-training	behavior transfer; capability recovery; cheap RL substitute
Black-box (§26.5)	teacher's	one token id (~17 bits)	cross-entropy (SFT)	API is enough	SFT stage	bootstrapping instruction data; any-tokenizer transfer

METHOD	WHOSE TEXT	SIGNAL PER TOKEN	LOSS	TEACHER ACCESS	PIPELINE STAGE	BEST AT
Reasoning traces (§26.5)	teacher's, filtered	one token id + a verified answer	cross-entropy (SFT)	API is enough	SFT stage	transferring reasoning to small models

The table's third column deserves a picture, because signal density is the quiet variable behind every cost claim in this chapter. Take one 512-token episode of student experience and ask: how much supervision does each framework extract from it?

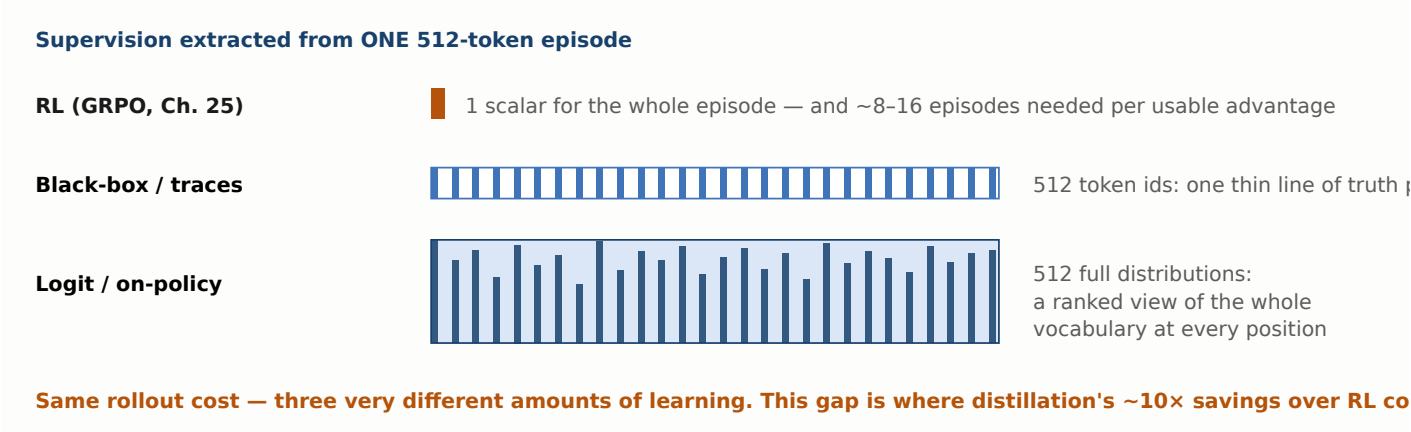


Figure 26.6 — Signal density per episode. The student pays the same autoregressive generation cost in all three frameworks (or none, for corpus-based logit KD). What differs is how much supervision comes back: a scalar, a token id per position, or a distribution per position. Dense signal converges in fewer episodes — the arithmetic behind Qwen3’s reported ~10× GPU-hour saving over RL.²

26.7 What the big labs do

Table 26.3 — Distillation in shipped model families.

MODEL FAMILY	METHOD	TEACHER → STUDENT	REPORTED OUTCOME
DeepSeek R1-Distill ¹	reasoning traces (800K, filtered)	R1 (671B MoE) → Qwen/Llama 1.5B–70B	32B: 72.6 AIME vs. 47.0 for direct RL; released openly
Qwen3 ²	off-policy warm start + on-policy logit distillation	235B MoE flagship → 0.6B–14B	~1/10 the GPU-hours of the RL pipeline, better pass@1/pass@k
Gemma 2 / 3 ³	logit distillation through pre-training	larger Gemma → 2B/9B (8T tokens for 9B)	small models "punch above" their token budget; 2B-class leader on release

MODEL FAMILY	METHOD	TEACHER → STUDENT	REPORTED OUTCOME
Llama 3.2 1B/3B ⁴	prune (width) + logit distillation (Ch. 22 recipe)	Llama 3.1 8B → 1B/3B	on-device models recovering most 8B capability
Minitron ⁸	prune + logit distillation	Nemotron 15B → 8B/4B	up to 40× fewer tokens than from-scratch; ~1.8× family cost saving
Phi-4 ¹¹	black-box (synthetic curricula)	GPT-4-class teacher → 14B	frontier-adjacent reasoning at 14B, mostly synthetic tokens
Zephyr ¹²	black-box SFT + distilled preferences (dDPO)	GPT-4/teacher ensemble → 7B	early proof that alignment itself distills
This book's run (Ch. 29)	planned: traces + on-policy for the small serving tier	70B → 7B-class	numbers to appear with the case study

Read the table column-wise and the pattern of §26.1 reappears: everyone who owns a big teacher uses logits (Google, Meta, NVIDIA, Alibaba); everyone whose teacher is an API uses text (Phi, Zephyr, the fine-tuning ecosystem); and the reasoning wave of 2025 runs on traces because traces are the only signal that crosses both the API boundary and the tokenizer boundary. Notice too what nobody ships anymore: small models post-trained by full RL from scratch. That column lost on cost *and* on quality, and Table 26.1 is why.

26.8 Practical considerations

Licensing before compute. For black-box distillation the binding constraint is legal, not technical. Frontier API terms of service restrict using outputs to train competing models; open-weight licenses differ on distillation specifically (some permit it with attribution or naming requirements, some restrict commercial use). This is not a footnote: it decides which teachers exist for you. Involve whoever owns legal risk *before* generating 800K traces, keep provenance metadata on every synthetic sample (teacher, date, prompt), and prefer teachers whose licenses explicitly bless distillation. R1's permissive license is a large part of why its distillates spread so fast.¹

The tokenizer wall. Logit and on-policy distillation compare distributions over a shared vocabulary at shared token boundaries. A Llama teacher cannot logit-distill into a Qwen student — different vocabularies, different segmentations, the loss is undefined. Your options, in descending order of sanity: pick a same-tokenizer pair (why distillation families stay in-family); fall back to black-box (text crosses any tokenizer); or reach for research bridges (universal/cross-tokenizer distillation that aligns via text spans) and accept the complexity. Check this on day zero — it is the single most common plan-killer.

The capacity gap. Bigger teacher is not monotonically better. A 1B student distilled from a 405B teacher does *worse* than from a 70B in most published ablations: the giant's distributions encode distinctions the student cannot represent,

and forward KL punishes it for failing to. Practical rule: prefer a teacher 4–10× the student, or insert an intermediate model as a "teaching assistant" when the gap is larger. If you own several sizes, distill down the ladder, not off the top.

Temperature and top-k are quality knobs, not defaults. $\tau \in [1, 2]$ for logit KD is the usual sweet range (Figure 26.2 shows why τ amplifies dark knowledge; too high and you teach noise among absurd tokens). If you truncate the teacher's distribution to top-k to save bandwidth, renormalize and keep $k \geq 64$ — aggressive truncation quietly deletes the dark knowledge you came for.

How to evaluate a distillate. Three numbers, tracked together, tell you whether a distillation is working; any one alone will mislead. First, *task evals on your distribution* (Chapter 9's suite) — the only number that pays rent. Second, the *win-rate against the teacher* on held-out prompts, judged blind: it tells you how much of the gap you have closed, and a distillate that plateaus at a low win-rate with a still-falling KD loss is matching easy mass, not capability (the pitfall below). Third, the *KL to the teacher on held-out student samples* — cheap to compute during training and the closest thing to a validation loss this procedure has; a rising held-out KL with a falling training KL is distillation's version of overfitting, and it shows up first on prompts far from the training mix. Run all three per language where that matters, and keep the teacher's own scores on the same dashboard: the ceiling is part of the measurement.

You distill flaws too. The teacher's over-refusals (Chapter 28), sycophancy, format quirks, and factual errors are all in its distributions and its samples, and the student will learn them with the same efficiency it learns everything else. Filter black-box data against these (Chapter 27's judges), and for logit/on-policy runs, evaluate the student on refusal calibration and bias suites — not only on capability — before you call the run a success.

Common pitfalls

Distilling into a cold student.

Pure on-policy from a random-ish student wastes the teacher on grading gibberish. Warm-start with SFT on teacher samples first (the Qwen3 order), then go on-policy.

Trusting the loss curve.

KD loss can fall while generation quality stalls — the student may be matching easy mass (stopwords, formatting) everywhere. Track task evals and win-rates against the teacher, not KL alone.

Grading prompt tokens.

Masking bugs that include the prompt in the KD loss teach the student to imitate the teacher on text it will never generate, and dilute the signal. Loss on completion tokens only (the `completion_mask` in §26.4's code).

Storing teacher logits "to save compute."

77 TB per 100B tokens at top-128 (§26.3). Run the teacher online; engineer its inference like serving.

Benchmark contamination via the teacher.

If the teacher saw a benchmark, its traces leak it, and your student's score is fiction. Decontaminate synthetic data against your eval suite (Chapter 10) exactly as you would web data.

Evaluating only on the teacher's home turf.

Trace-distilled reasoners imitate the shape of the teacher's reasoning; test out-of-distribution (your domain, your languages — Spanish and Portuguese, for us) before shipping.

26.9 Decision framework

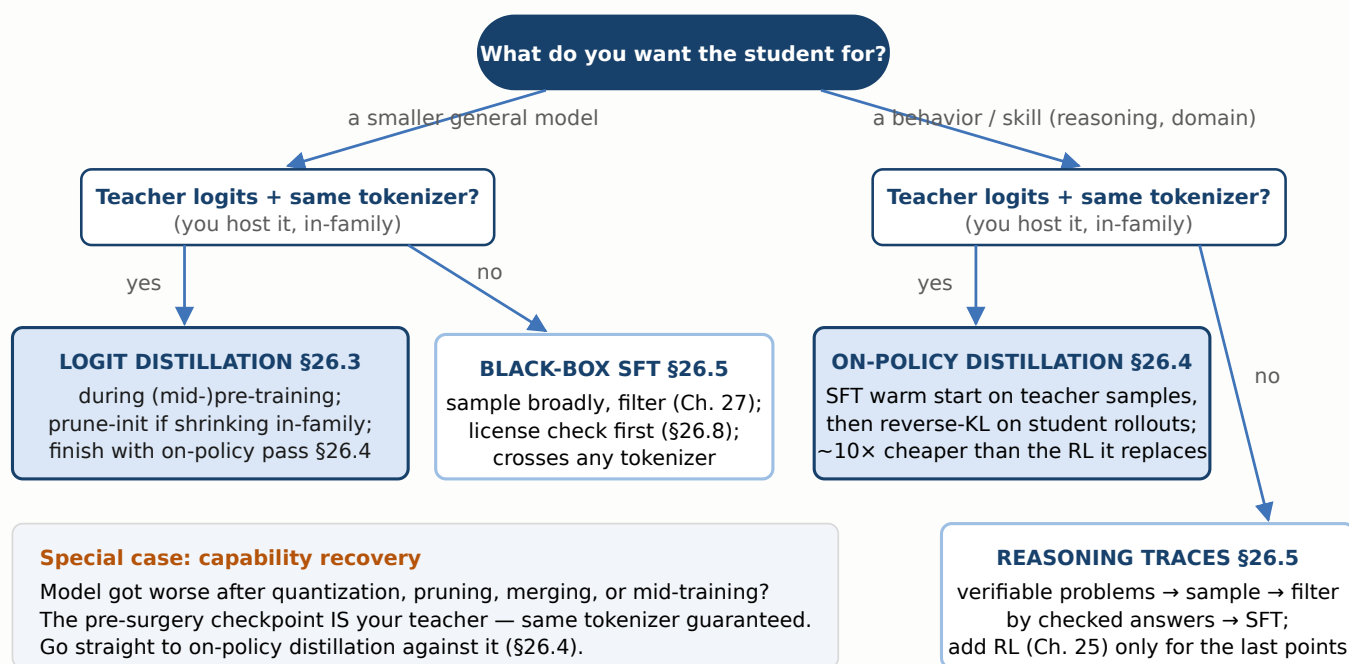


Figure 26.7 — Choosing a distillation. Two questions route every case: what the student is for, and whether the teacher's logits are readable through a shared tokenizer. The special case at bottom-left is the one that surprises teams: after any surgery on a model, distillation from its own former self is the cheapest repair — and it always passes the access check.

Cost-benefit, bluntly. Reasoning traces: API budget and filtering effort, no training-infra beyond SFT — the highest capability-per-dollar move in this book when the skill is verifiable, which is why it is the default answer for "make my 7B reason." Black-box SFT: same cost profile, broader but shallower gains; the license check is the real price. On-policy distillation: needs the teacher served next to the trainer — real infrastructure — but replaces an RL pipeline that needs the same rollouts *plus* reward models, group sampling, and hacking-patrol, at ~10× the GPU-hours.² Logit distillation through pre-training: the most expensive (it doubles per-token cost against a same-family teacher) and the most valuable per point of quality when you are building a model family anyway — you were going to pay for the small model's tokens; paying ~2× for 40×-fewer of them is the bargain Minitron quantified.⁸

How this scales. The teacher's cost is inference, so everything Chapter 13 did to make inference cheap — GQA, MLA, quantized serving — directly lowers the price of distillation; a 4-bit teacher grades nearly as well for a quarter of the bandwidth. The student side scales the other way: the smaller the student, the larger the share of your quality that must come from the teacher's signal rather than the student's own compute, so the *relative* payoff of distillation grows as you shrink. That is why the bottom of every modern model family is distilled and the top never is — and why, on a multi-

size roadmap, the right mental model is one expensive flagship run (Parts III–IV) followed by distillation as the *manufacturing process* for every other size.

26.10 Summary

Distillation is one idea — train a model on another model's outputs — instantiated four ways, and the differences are decided by two questions: what the teacher emits, and whose text gets graded. Logit distillation streams the teacher's full distributions over a fixed corpus (forward KL, τ -softened): the signal-densest way to build a small model, the engine of Gemma's and Llama 3.2's small tiers, and the healing step after Chapter 22's pruning — at the price of a hosted, same-tokenizer teacher and roughly doubled per-token cost. On-policy distillation has the teacher grade every token of the student's own samples with reverse KL: dense supervision exactly where the student's errors live, no reward models or group rollouts, and the published $\sim 10\times$ GPU-hour savings over the RL it replaces — the modern default for behavior transfer and for repairing a model after quantization, pruning, or merging (its own former checkpoint is the teacher). When only text crosses the boundary, black-box distillation is synthetic-data SFT under another name, and reasoning-trace distillation is its sharpest form: sample chains of thought, keep the verifiably correct, SFT — the R1 recipe that beat direct RL by 25 AIME points on the same student. Choose by access and goal (Figure 26.7), check the tokenizer and the license before anything else, distill from a teacher 4– $10\times$ the student, warm-start before going on-policy, and evaluate the student on your distribution — including the flaws it may have inherited. In a book about efficiency, this chapter is the closing argument: the cheapest capability you will ever add to a small model is capability someone already paid to put in a big one.

References

1. DeepSeek-AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. Distilled models from 800K curated samples (600K reasoning); R1-Distill-Qwen-32B 72.6 AIME / 94.3 MATH-500 vs. 47.0 / 91.6 for direct large-scale RL on the same base; R1-Distill-Qwen-7B 55.5 AIME / 92.8 MATH-500.
2. Qwen Team (2025). Qwen3 Technical Report. arXiv:2505.09388. Strong-to-weak distillation (off-policy warm start + on-policy logit distillation) for the 0.6B–14B tier; reported at roughly 1/10 the GPU-hours of the RL pipeline with better pass@1/pass@k.
3. Gemma Team, Google DeepMind (2024). Gemma 2: Improving Open Language Models at a Practical Size. arXiv:2408.00118. 2B and 9B trained with knowledge distillation from a larger teacher; 9B distilled over 8T tokens.
4. Meta AI (2024). Llama 3.2: Revolutionizing edge AI and vision with open models. Meta AI blog, Sept 2024. 1B/3B built by width pruning Llama 3.1 8B and recovering with logit distillation.
5. Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. arXiv:1503.02531. Soft targets, temperature, the τ^2 gradient-scale correction, and "dark knowledge".
6. Gu, Y., Dong, L., Wei, F., & Huang, M. (2024). MiniLLM: Knowledge Distillation of Large Language Models. ICLR. arXiv:2306.08543. Reverse KL for generative students; mode-seeking beats mass-covering for open-ended

generation.

7. Agarwal, R., et al. (2024). On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes (GKD). ICLR. arXiv:2306.13649. Trains on student-generated sequences against the teacher's per-token distributions; frames exposure bias as the train–inference distribution mismatch.
8. Muralidharan, S., et al. (2024). Compact Language Models via Pruning and Knowledge Distillation (Minitron). NeurIPS. arXiv:2407.14679. 15B → 8B/4B with up to 40× fewer training tokens than from-scratch; ~1.8× compute saving for the family.
9. Thinking Machines Lab (2025). On-Policy Distillation. thinkingmachines.ai/blog, Oct 2025. Reproduction and cost analysis of on-policy distillation for reasoning and capability recovery; reports order-of-magnitude (~10×) compute reductions vs. RL on matched tasks. (Blog; treat numbers as the authors' estimates.)
10. Kim, Y., & Rush, A. M. (2016). Sequence-Level Knowledge Distillation. EMNLP. arXiv:1606.07947. Distillation through the teacher's sampled sequences — the formal ancestor of black-box distillation.
11. Abdin, M., et al. (2024). Phi-4 Technical Report. arXiv:2412.08905. 14B trained predominantly on synthetic data generated by frontier teachers, with heavy curation.
12. Tunstall, L., et al. (2023). Zephyr: Direct Distillation of LM Alignment. arXiv:2310.16944. Distilled SFT plus distilled DPO from teacher-ranked preferences.

Next: *Chapter 27 — Synthetic data*, the generation-and-filtering machinery this chapter kept borrowing; then *Chapter 28 — Safety, refusals, and calibration*, including what distillation does to a model's refusal behavior. · [Training Efficient LLMs](#)